The 3×3 stride-1 convolution kernel

Om Tank · YOLOv8n on one Broadwell core, built like the Assignment 2 kernel

32
layers: C2f bottlenecks + Detect head

66%
of YOLOv8n's 8.74 GFLOP

94%
of peak: best microkernel alone

84%
of peak across all 32 real layers

It is the Assignment 2 kernel, with floats

	Assignment 2	Stride-1 microkernel
m (broadcast)	5 rows of A	5 output pixels
n (vector loads)	$8 = 2 \times 4$ doubles	16 channels $= 2 \times 8$ floats
k (reduction)	32 to 512	$C \times 9$ (144 to 2304)
Accumulators	10	10 (2×5) or 12 (2×6)
Registers	13 of 16	13 or 15 of 16
Peak per core	16 FLOPs/cycle	32 FLOPs/cycle

Same Broadwell core: 2 FMA units $\times$ 8 lanes $\times$ 2 FLOPs = 32 FLOPs/cycle. Latency 5 $\times$ throughput 2 = 10 chains in flight.

`C`

input channels of the layer (16 to 256)

`k = C x 9`

multiply-adds per output: C channels $\times$ the 9 taps of the 3×3 filter (A2's k)

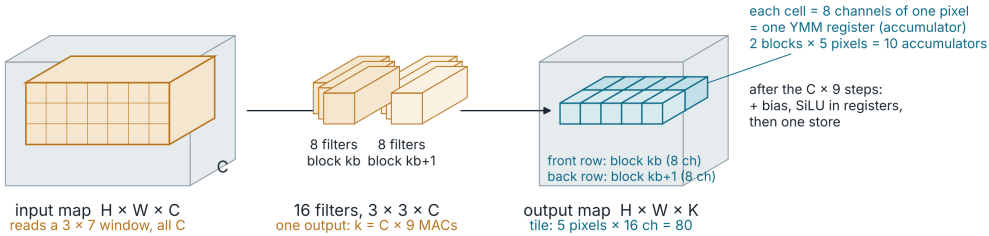
`accumulator`

one YMM register = 8 outputs (8 channels of one pixel); one dependency chain

`tile 2x5 (10)`

outputs per call: 2 blocks of 8 channels $\times$ 5 pixels = 80 outputs in 10 accumulators

One kernel call = 80 outputs

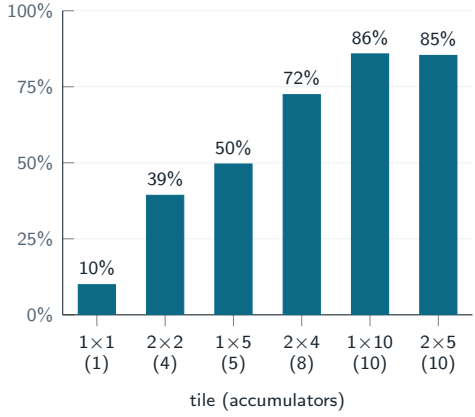


Per step (one input channel at one of the 9 taps): load 2 weight vectors, broadcast 5 inputs, 10 FMAs. Repeat $k = C \times 9$ times, then bias, SiLU and one store per register.

Four steps: ops → accumulators → tile → benchmark

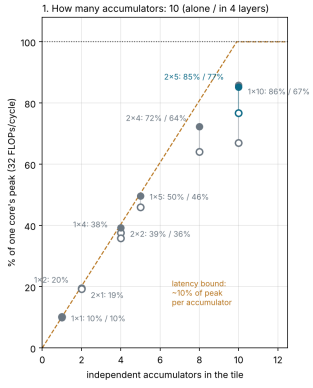
1. **Independent ops.** Every output is its own dot product of length $C \times 9$: 102,400 in a 40×40, 64-channel layer.
2. **Outputs in flight.** Latency 5 × 2 FMA/cycle = 10 accumulators = 80 outputs; 1 accumulator runs at 10% of peak.
3. **Kernel size.** 2×5 tile: 10 acc + 2 weights + 1 broadcast = 13 regs; 7 loads per 10 FMAs; 5 divides every width.
4. **Benchmark like A2.** Microkernel alone, $k = 72$ to 4608, 3 runs, 2 s warm-up, cycles at 3.4 GHz, checked against a reference.

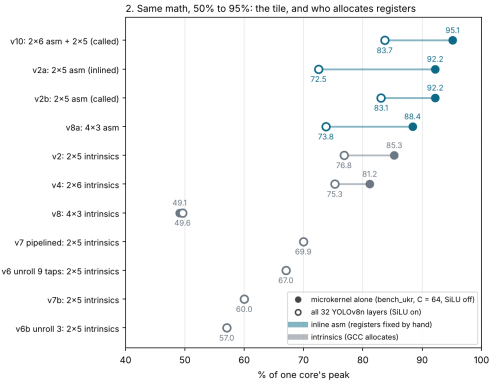
Microkernel alone vs accumulators ($C = 64$, SiLU off)



Same math, 50% to 95%: tile and register allocation

Stride-1 kernel design space on ece028: hand-allocated tiles reach 95% alone and 84% in all 32 layers





1 core, cycles = time × 3.4 GHz (max turbo, as in Assignment 2), GCC 8.5 -O3 -march=broadwell, FP32. Sources: results/s1_*_bl_t1.csv, s1_tile_sweep.csv, ukr_C64_nosilu_34.csv.

Intrinsics

GCC: clean at 13 regs, copies at 15, spills at 16

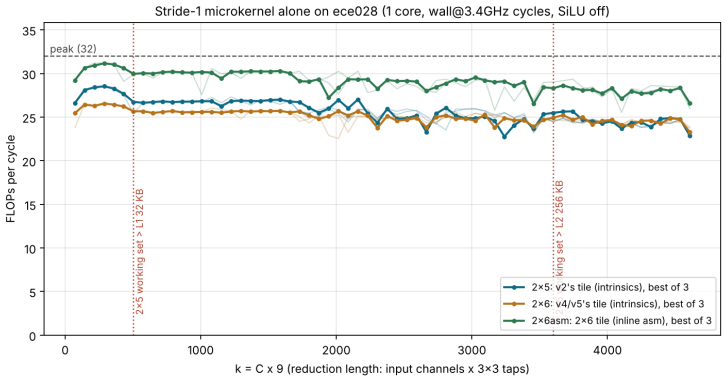
Inline asm

2×6 by hand: 95% alone, 84% in layers

Unroll, pipeline

lost: uop cache, spills

Benchmarked like Assignment 2: FLOPs/cycle vs k



SiLU off, cycles = time × 3.4 GHz, as in A2. 3 runs, every point checked (576 pass).

94–97%

2×6 asm: 30–31 FLOPs/cycle, ∼102 GFLOP/s on one core

83–89%

2×5 intrinsics (v2's tile)

$k \approx 500$

L1 dip: 2×5 weights pass 32 KB, like $k \approx 315$ in A2

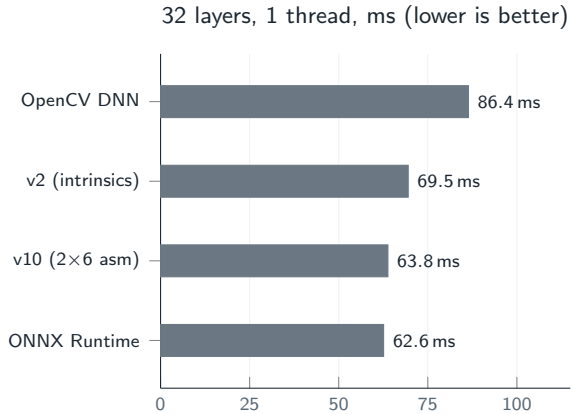
Alone vs in a layer: keep the kernel a function

v2a (2×5 asm): 92.2% alone, 72.5% when GCC inlines it into the layer loop

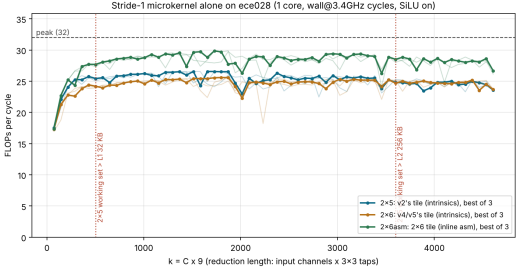
```
vmovaps 168(%rsp), %ymm1   # reload acc
  ...asm block: 80 FMAs...
vmovaps %ymm1, 168(%rsp)   # store acc
```

Inlined, GCC keeps 3 of 10 accumulators on the stack around every tap. Called as a function, as in the benchmark: 83.1% (v2b); with the 2×6 tile, 83.7% (v10).

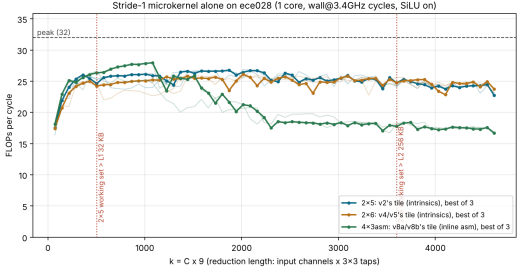
Next: cache blocking for $C \geq 128$ (the 4×3 drop-off), NUMA-aware threads (v10 on 20 cores: 4.74 ms).



Backup: the same sweeps with SiLU on



2×6 asm 90.2% mean ($C = 32$–256); 2×5 81.4%. SiLU costs a fixed 160–290 cycles per call.



4×3 asm: best 87.5% at $C = 120$, 57% at $C = 256$: its weights leave L2 (cache blocking fixes this).